

Everything Claude Code (ECC)

The Production-Grade Working Guide & Reference Framework

Author: Affaan Mustafa (@affaanmustafa)

Source: Anthropic Hackathon Winning Setup

This guide materializes the complete engineering framework open-sourced by Affaan Mustafa, based on his Anthropic Hackathon-winning project **Everything Claude Code (ECC)**. It details how to hardcode elite workflows directly into the CLI agent layer to transition from casual "vibe-coding" to production-grade, highly predictable autonomous development.

1. The Core Architecture Pillars

The ECC framework breaks down Claude Code configuration into six distinct conceptual building blocks. Understanding how these layers map onto the runtime environment prevents prompt bloat and keeps context tidy:

Building Block	Core Concept & Purpose	Deployment Scope & Context Impact
Skills	Reusable workflow definitions and procedural guidelines written in Markdown. Explicitly invoked or mapped to actions.	Stored globally in <code>~/.claude/skills/</code> . Highly targeted workflow activation.
Commands	Slash-style CLI triggers (e.g., <code>/plan</code> , <code>/tdd</code>) executing explicit backend instructions.	Stored in <code>~/.claude/commands/</code> . Drastically cuts down repetitive command text entry.
Rules	Uncompromising, persistent baseline rules (e.g., security, formatting) that Claude must always follow.	Hierarchical storage. Best kept modular and structured by specific concerns (e.g., language stacks).
Hooks	Event-driven automation triggered by tool execution thresholds (e.g., intercepting file edits or commits).	Configured via event matchers. Executes background scripts without agent friction.
MCPs	Model Context Protocol extensions connecting Claude directly to external APIs or local tool pipelines.	Configured via JSON. Directly affects baseline context length based on tool counts.

Building Block	Core Concept & Purpose	Deployment Scope & Context Impact
Plugins	Unified distribution format that packages multiple skills, hooks, and MCP tools together.	Managed using the marketplace CLI namespace (e.g., <code>/ecc:plan</code>).

2. Advanced Memory System (CLAUDE.md Structure)

Claude Code relies on a multi-tiered file-based hierarchy to persist system-wide and workspace-level context. Instead of treating memory as a monolithic dump, structure it into three clear tiers:

- **Project Level (`./CLAUDE.md`):** Shared with your team across the repository. Contains specific build commands, test patterns, framework versions, and project rules.
- **User Level (`~/.claude/CLAUDE.md`):** Personal baseline settings applied across all local coding workspaces on your computer.
- **Project-Specific User Level (`.claude/CLAUDE.md`):** Project overrides tailored to your personal developer machine environment.

The Instinct Integration Pattern

ECC supports exporting runtime session historical insights into localized `.md` Instinct files. They capture empirical corrections observed during interactive programming loops, structured with explicit blocks: `# Action` , `# Evidence` , and `# Examples` .

3. Context Optimization & Resource Hardening

When scaling autonomous loops across large codebases, context window saturation is your greatest enemy. Stacking too many active tools will shrink your effective reasoning window. Follow these mandatory optimization rules:

The Three Constraints For Resource Optimization

- **The 40% Baseline Limit:** Ensure initial project `/context` ingestion consumes less than 40% of the total context limits. For reference, a typical monorepo baseline structure uses roughly 20K tokens (~10%). Exceeding 40% demands directory exclusion tuning.
- **The 10-MCP Ceiling:** Keep 10 or fewer MCPs active simultaneously per project workspace. Active MCP tool descriptions consume context constantly. Exceeding this risks shrinking your available context window down to 70K. Keep active tools under 80 total.
- **The 2% Skill Cap:** System skill descriptions are aggressively parsed up to a rigid limit of 2% of the context. If you see active "skill exclusion warnings" in your terminal output, you must refactor your custom markdown workflows immediately.

The Anti-Inflation Rule (Overusing 'IMPORTANT'): As explicit directive frequency increases, uniform model adherence degrades. Avoid packing systems with redundant `IMPORTANT` flags. Instead, delegate heavy tasks into granular execution layers using **Subagents** to isolate scope.

4. Workflow Implementation & Shell Setup

Keyboard & Terminal Navigation Shortcuts

Shortcut Key	Execution & Interface Action
<code>ctrl + u</code>	Delete the entire active terminal text line instantly (prevents backspace spam).
<code>!<command></code>	Escape prefix to run an absolute local bash command directly from inside the agent terminal.
<code>@</code>	Global project file search autocomplete catalog list trigger.
<code>/</code>	Initiates slash workspace macros and plugin actions.
<code>Shift + Enter</code>	Inserts a multi-line newline without firing the active prompt stream.
<code>Tab</code>	Toggles the detailed view of Claude's background thinking and tool arguments.
<code>Esc Esc</code>	Instantly interrupts active generations or initiates emergency context restoration.

Parallel Execution Frameworks

To avoid queuing blocking tasks, execute concurrent operations using two distinct strategies:

1. **The Conversation Fork:** Use the `/fork` command to split conversations for non-overlapping isolated tasks.
2. **Git Worktrees:** For overlapping operations, create isolated feature sandboxes so concurrent agent instances work independently without conflict:

```
# Create isolated worktrees for concurrent agent processes
git worktree add ../feature-auth -b feature/auth
git worktree add ../feature-payments -b feature/payments

# Execute individual Claude Code sessions inside each targeted worktree folder
cd ../feature-auth && claude
```

5. Installation & Verification Loop

Plugin Activation & Rules Integration

ECC can be handled natively as a namespace plugin or deployed via local configuration rules:

```
# Option 1: Native Plugin Installation Route
/plugin marketplace add https://github.com/affaan-m/ECC
/plugin install ecc@ecc

# Option 2: Manual Language Rule Injection
git clone https://github.com/affaan-m/ECC.git && cd ECC
# Target explicit language stacks to preserve context
./install.sh typescript python golang
```

Critical Installation Rule

Do not run the full manual installer (`./install.sh --profile full`) after running a plugin command. This will copy duplicate skills into your user folders, causing runtime collision. For plugin configurations, copy only targeted `rules/` directories to your local machine path (`~/.claude/rules/ecc/`).

The Verification Loop Lifecycle

Enforce strict operational criteria during code generation by running systematic tracking loops coupled with security gateways:

```
# Begin an explicitly tracked validation loop
/ecc:plan "Implement JWT verification hook"
```

```
/everything-claude-code:loop-start

# Execute Test-Driven Development loops (Target 80%+ Test Coverage)
# Run security analysis via the localized AgentShield test suite
/everything-claude-code:harness-audit
/everything-claude-code:quality-gate
```

The automated `AgentShield` verification engine executes up to 1,282 integrated configuration validation tests across 102 rule definitions to flag injection exposure vulnerabilities, hardcoded secrets, or unvalidated inputs before code hits your git commit stream.